此次练习中，我们使用Human Activity Recognition Using Smartphones数据集。它通过对参加测试者的智能手机上安装一个传感器而采集了参加测试者每天的日常活动（ADL）。目标是将日常活动分成六类（walking， walking upstairs， walk downstairs， sitting， stand， and laying）。

该数据集也可以在Kaggle网站上获得：https://www.kaggle.com/uciml/human-activity-recognition-with-smartphones/downloads/human-activity-recognition-with-smartphones.zip

把训练文件重新命名为 Human_Activity_Recognition_Using_Smartphones_Data.csv

第一步：导入数据

- 查看数据类型---因为有太多的列，所以最好使用value_counts
- 判断其中的小数数值是否需要尺度缩放
- 检查数据中各活动类型的划分
- 把活动类型标签编码成一个整数

在 [11]…

```
import pandas as pd
import numpy as np

filepath = 'Human_Activity_Recognition_Using_Smartphones_Data.csv'
data = pd.read_csv(filepath)
```

所有列的数据类型都是浮点数，除了活动标签列。

在 [3] …

```
data.dtypes.value_counts()
```

输出[3]:

```
float64    561
int64        1
object       1
Name: count, dtype: int64
```

在 [4] …

```
data.dtypes.tail()
```

输出[4]:

```
angle(X,gravityMean)    float64
angle(Y,gravityMean)    float64
angle(Z,gravityMean)    float64
subject                   int64
Activity                 object
dtype: object
```

数据都已经全部被缩放到-1到1之间了。

在 [12]…

```
data.iloc[:, :-1].min().value_counts()
```

出[12]:

```
-1.000000    466
-0.995377      2
-0.999996      2
-0.999893      2
-1.000000      2
            ...
-0.999983      1
-0.943439      1
-0.998014      1
-0.999915      1
 1.000000      1
Name: count, Length: 93, dtype: int64
```

在 [13]…

```
data.iloc[:, :-1].max().value_counts()
```

出[13]:

```
1.000000     452
0.994731       2
0.805064       1
0.908361       1
0.891736       1
            ...
0.990935       1
0.979031       1
0.928416       1
0.848031       1
30.000000      1
Name: count, Length: 110, dtype: int64
```

检查数据中各活动类型的划分---已经比较平衡了。

在 [14]…

```
data.Activity.value_counts()
```

输出[14]…

```
Activity
LAYING                1407
STANDING              1374
SITTING               1286
WALKING               1226
WALKING_UPSTAIRS      1073
WALKING_DOWNSTAIRS     986
Name: count, dtype: int64
```

Scikit learn的分类器不接受一个稀疏矩阵作为预测列。所以，可以使用将活动标签编码为整数。 `LabelEncoder`

在 [15]…

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
data['Activity'] = le.fit_transform(data.Activity)
data['Activity'].sample(5)
```

出局[15]…

```
7176    2
2164    3
831     5
1264    3
458     3
Name: Activity, dtype: int32
```

第二步：划分训练数据和测试数据

- 可以考虑使用Scikit-learn中的，以保证划分后的数据集中每个类别个案的比例与整个数据集相同。 `StratifiedShuffleSplit`

在 [16]…

```
feature_cols = data.columns[:-1]

from sklearn.model_selection import StratifiedShuffleSplit

# Get the split indexes
strat_shuf_split = StratifiedShuffleSplit(n_splits=1,test_size=0.3, random_state

train_idx, test_idx = next(strat_shuf_split.split(data[feature_cols], data.Activ

# Create the dataframes
X_train = data.loc[train_idx, feature_cols]
y_train = data.loc[train_idx, 'Activity']

X_test  = data.loc[test_idx, feature_cols]
y_test  = data.loc[test_idx, 'Activity']
```

在 [17]…

```
y_train.value_counts(normalize=True)
```

出[17]:

```
Activity
0    0.191411
2    0.186941
1    0.174893
3    0.166731
5    0.145939
4    0.134085
Name: proportion, dtype: float64
```

在 [18]…

```
y_test.value_counts(normalize=True)
```

出[18]:

```
Activity
0    0.191296
2    0.186763
1    0.174977
3    0.166818
5    0.145966
4    0.134180
Name: proportion, dtype: float64
```

第三步：训练模型

- 用所有特征训练一个基本的使用缺省参数的逻辑回归模型。
- 分别用L1和L2正则化来训练一个模型，使用交叉验证确定超参数的值。注意，正则化模型，尤其是L1模型可能需要一定训练时间。

在 [21]…

```
# 请在此处填写你的代码（训练一个基本的使用缺省参数的逻辑回归模型）
from sklearn.linear_model import LogisticRegression

# 训练基本的逻辑回归模型
lr = LogisticRegression(solver='liblinear').fit(X_train, y_train)
```

在 [22]…

```
from sklearn.linear_model import LogisticRegressionCV
```

```
# L1 正则化的逻辑回归
lr_l1 = LogisticRegressionCV(Cs=10, cv=4, penalty='l1', solver='liblinear').fit(
```

在 [23]…

```
# 请在此处填写你的代码（L2 正则化的逻辑回归）
from sklearn.linear_model import LogisticRegressionCV

# L2 正则化的逻辑回归
lr_l2 = LogisticRegressionCV(Cs=10, cv=4, penalty='l2', solver='liblinear').fit(
```

第四步：

- 输出上面训练出的三个模型中每个特征的系数；
- 并绘制成图来比较它们的差异 （每个类别一张图）

In [25]:

```
# 请在此处填写你的代码（输出各模型训练到的特征系数值）
import pandas as pd

# 基本逻辑回归模型的系数
lr_coef = pd.DataFrame(lr.coef_, columns=feature_cols)
# L1 正则化逻辑回归模型的系数
lr_l1_coef = pd.DataFrame(lr_l1.coef_, columns=feature_cols)
# L2 正则化逻辑回归模型的系数
lr_l2_coef = pd.DataFrame(lr_l2.coef_, columns=feature_cols)

print("基本逻辑回归模型系数：")
print(lr_coef)
print("L1 正则化逻辑回归模型系数：")
print(lr_l1_coef)
print("L2 正则化逻辑回归模型系数：")
print(lr_l2_coef)
```

基本逻辑回归模型系数：

```
   tBodyAcc-mean()-X  tBodyAcc-mean()-Y  tBodyAcc-mean()-Z  tBodyAcc-std()-X  \
0          -0.086613           0.004251           0.053344          0.000762
1           0.040935          -0.008299          -0.099716         -0.556278
2           0.014581           0.085078           0.027654         -0.349247
3          -0.123309          -0.055487           0.123161         -0.609670
4           0.493333           0.103618           0.156967          0.345477
5          -0.245626          -0.314549          -0.267448          0.637585

   tBodyAcc-std()-Y  tBodyAcc-std()-Z  tBodyAcc-mad()-X  tBodyAcc-mad()-Y  \
0         -0.055257          0.022439          0.004497         -0.058661
1          0.009038         -0.303799         -0.511620         -0.034934
2         -0.439479          0.118469         -0.341971         -0.344521
3          0.274185          0.011680         -0.465197          0.268738
4         -0.414391         -0.320035          0.376417         -0.358737
5          0.083071          0.206236          0.190666          0.052414

   tBodyAcc-mad()-Z  tBodyAcc-max()-X  ...  fBodyBodyGyroJerkMag-skewness()  \
0          0.028239         -0.094497  ...                         -0.216596
1         -0.237907         -0.560053  ...                          0.349483
2          0.133786         -0.357532  ...                         -0.421767
3         -0.061661         -0.651919  ...                          0.382859
4         -0.282657          0.979117  ...                         -0.863845
5          0.318877          0.541914  ...                          0.058257

   fBodyBodyGyroJerkMag-kurtosis()  angle(tBodyAccMean,gravity)  \
0                        -0.251948                    -0.076362
1                         0.039253                     0.305386
2                         0.125064                    -0.296907
3                         0.332013                     0.058198
4                        -0.857771                     0.031442
5                         0.455632                     0.028739

   angle(tBodyAccJerkMean),gravityMean)  angle(tBodyGyroMean,gravityMean)  \
0                              0.018485                         -0.112378
1                             -0.062932                         -0.504581
2                              0.033254                          0.444733
3                              0.087430                         -0.202963
4                             -0.249687                          0.337847
5                             -0.008977                         -0.264407

   angle(tBodyGyroJerkMean,gravityMean)  angle(X,gravityMean)  \
0                              0.006223              1.551960
1                             -0.243453             -1.720499
2                              0.303618             -1.006437
3                             -0.083863             -0.321189
4                              0.011228             -0.567405
5                             -0.044048              0.455930

   angle(Y,gravityMean)  angle(Z,gravityMean)   subject
0             -0.364216             -0.179551 -0.049807
1             -1.803703             -0.195800  0.024035
2              2.007979              0.229937 -0.035159
3             -0.197140              0.081646  0.003455
4             -0.344011             -0.084978  0.007241
5              0.845259              0.366972 -0.007094

[6 rows x 562 columns]
```

L1 正则化逻辑回归模型系数：

```
   tBodyAcc-mean()-X  tBodyAcc-mean()-Y  tBodyAcc-mean()-Z  tBodyAcc-std()-X  \
```

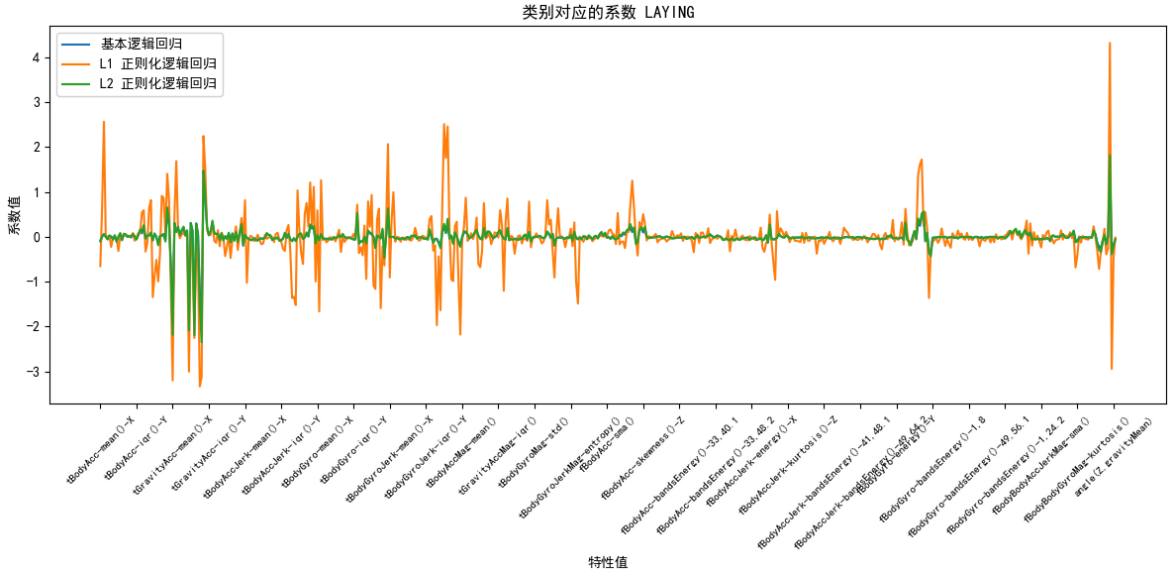
```
0          -0.653395           0.647194           2.563480           0.048065
1           0.000000           0.000000           0.000000          -1.646047
2           0.000000           0.000000           0.000000          -0.124372
3          -1.412605         -10.061538           1.483833          -1.702223
4           2.389671           0.000000           0.000000           0.154910
5          -2.134326          -8.458475          -3.802418           2.134650

   tBodyAcc-std()-Y  tBodyAcc-std()-Z  tBodyAcc-mad()-X  tBodyAcc-mad()-Y  \
0         -0.030216          0.028409         -0.225061         -0.026428
1          0.000000         -1.540649         -1.284529          0.000000
2         -1.240854          0.000000         -0.045140         -0.694411
3          0.974054         -0.054937         -0.342269          1.173257
4         -1.244097          0.000000          0.573973          0.000000
5          0.000000          0.144030          0.425813         -0.012387

   tBodyAcc-mad()-Z  tBodyAcc-max()-X  ...  fBodyBodyGyroJerkMag-skewness()  \
0         -0.020358         -0.028293  ...                         -0.716367
1         -1.520856         -2.403233  ...                          1.187138
2          0.000000         -0.819836  ...                         -0.713140
3         -0.003900         -1.207721  ...                          1.128606
4         -0.295687          2.768459  ...                         -2.528031
5          0.948530          0.131511  ...                          0.982912

   fBodyBodyGyroJerkMag-kurtosis()  angle(tBodyAccMean,gravity)  \
0                        -0.342898                    -0.120943
1                         0.000000                     2.470437
2                         0.000000                    -0.256053
3                         0.107375                     0.465375
4                        -0.628382                     0.006979
5                         0.508532                    -0.360549

   angle(tBodyAccJerkMean),gravityMean)  angle(tBodyGyroMean,gravityMean)  \
0                              0.177232                         -0.391527
1                             -0.273037                         -0.295755
2                              0.155731                          0.344735
3                              0.274316                         -0.378250
4                             -0.267429                          0.444199
5                              0.006767                         -0.532784

   angle(tBodyGyroJerkMean,gravityMean)  angle(X,gravityMean)  \
0                             -0.259582              4.319172
1                             -1.071930             -4.821620
2                              0.803182             -1.799237
3                             -0.325657             -0.475534
4                              0.190839             -0.513939
5                              0.233452              0.342238

   angle(Y,gravityMean)  angle(Z,gravityMean)   subject
0             -2.944686             -0.467778 -0.017867
1              0.000000              0.000000  0.091410
2              0.000000              0.000000 -0.051156
3             -0.258765              0.000000 -0.002148
4              0.000000              0.000000 -0.001998
5              3.066002              2.978138  0.015456

[6 rows x 562 columns]
L2 正则化逻辑回归模型系数:
   tBodyAcc-mean()-X  tBodyAcc-mean()-Y  tBodyAcc-mean()-Z  tBodyAcc-std()-X  \
0          -0.103726           0.005506           0.063348          0.001230
1           0.105759          -0.117718          -0.303955         -0.807325
```

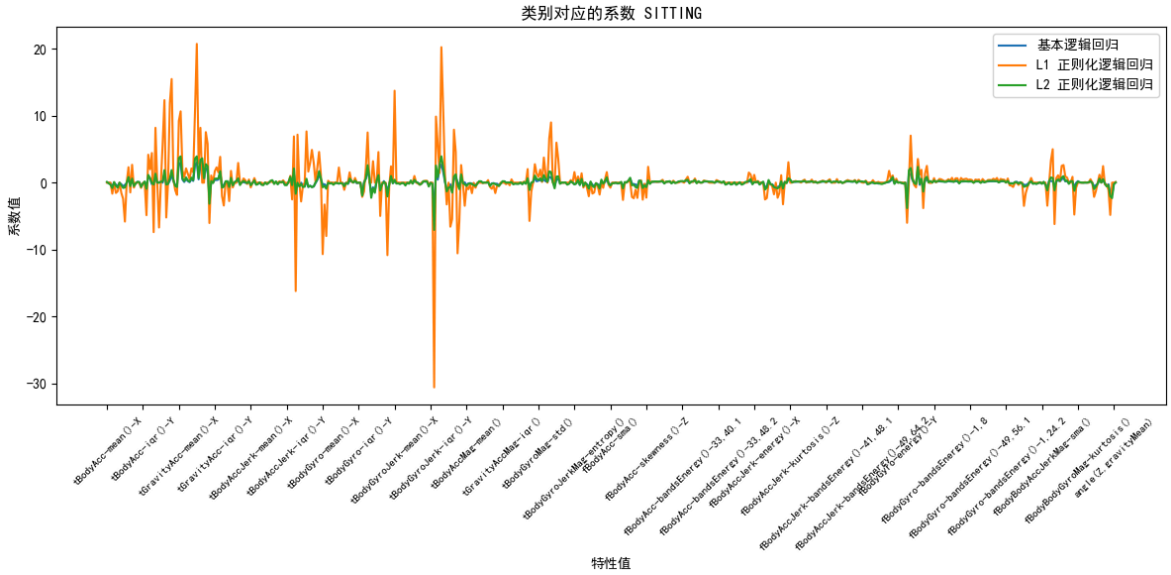
```
2           0.008618           0.240843           0.125793          -0.495693
3          -0.309551          -0.146430           0.288589          -1.141194
4           0.671721           0.143033           0.224426           0.385534
5          -0.468801          -0.734171          -0.865870           1.586454

   tBodyAcc-std()-Y  tBodyAcc-std()-Z  tBodyAcc-mad()-X  tBodyAcc-mad()-Y  \
0         -0.057114          0.028367          0.005432         -0.062392
1          0.095709         -0.487398         -0.737216          0.006312
2         -0.632435          0.259090         -0.494283         -0.437604
3          0.600326          0.000802         -0.889117          0.575397
4         -0.503176         -0.416957          0.436840         -0.386397
5          0.080839          0.454798          0.440742         -0.129012

   tBodyAcc-mad()-Z  tBodyAcc-max()-X  ...  fBodyBodyGyroJerkMag-skewness()  \
0          0.034581         -0.112413  ...                        -0.290982
1         -0.389462         -0.788929  ...                         0.601145
2          0.335523         -0.407519  ...                        -0.712661
3         -0.163550         -1.039752  ...                         0.620052
4         -0.351026          1.205335  ...                        -1.065623
5          0.763957          0.789511  ...                        -0.127402

   fBodyBodyGyroJerkMag-kurtosis()  angle(tBodyAccMean,gravity)  \
0                        -0.336144                    -0.117675
1                         0.012860                     0.531976
2                         0.155205                    -0.574802
3                         0.472525                     0.222474
4                        -1.043492                     0.043600
5                         1.060104                    -0.052867

   angle(tBodyAccJerkMean),gravityMean)  angle(tBodyGyroMean,gravityMean)  \
0                              0.044145                         -0.172642
1                             -0.123634                         -0.561500
2                              0.072111                          0.490018
3                              0.146631                         -0.367246
4                             -0.326418                          0.384248
5                             -0.188464                         -0.504442

   angle(tBodyGyroJerkMean,gravityMean)  angle(X,gravityMean)  \
0                              0.012597              1.819253
1                             -0.380067             -2.038317
2                              0.475306             -1.494627
3                             -0.186580             -0.608912
4                              0.025116             -0.739071
5                             -0.057955              1.036236

   angle(Y,gravityMean)  angle(Z,gravityMean)   subject
0             -0.388543             -0.192209 -0.058490
1             -2.313784             -0.232402  0.034390
2              2.543685              0.331423 -0.042676
3             -0.462981              0.134360  0.001331
4             -0.458979             -0.102601  0.008061
5              1.868783              0.799876  0.015841

[6 rows x 562 columns]
```

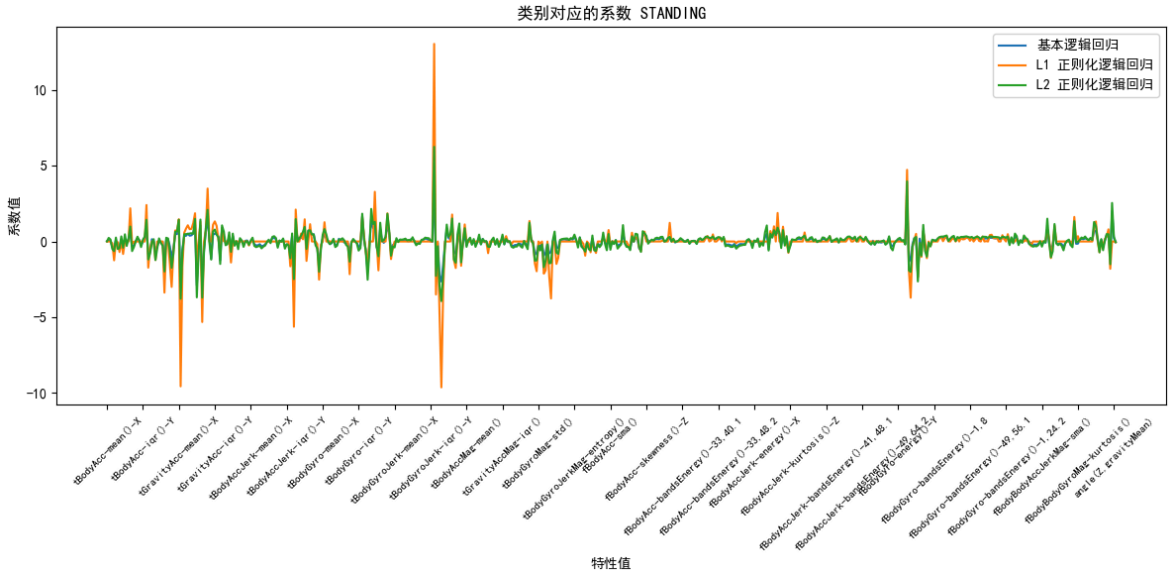
在 [39]…

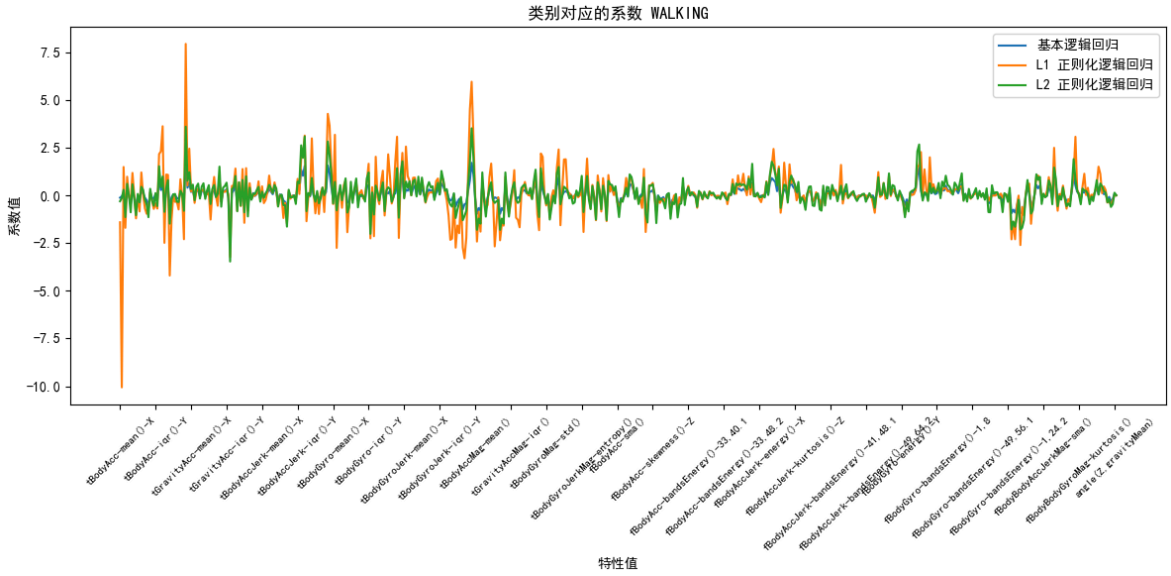
```python
# 请在此处填写你的代码（绘制6张图）
import matplotlib.pyplot as plt

# 活动类别数
num_classes = len(le.classes_)
```

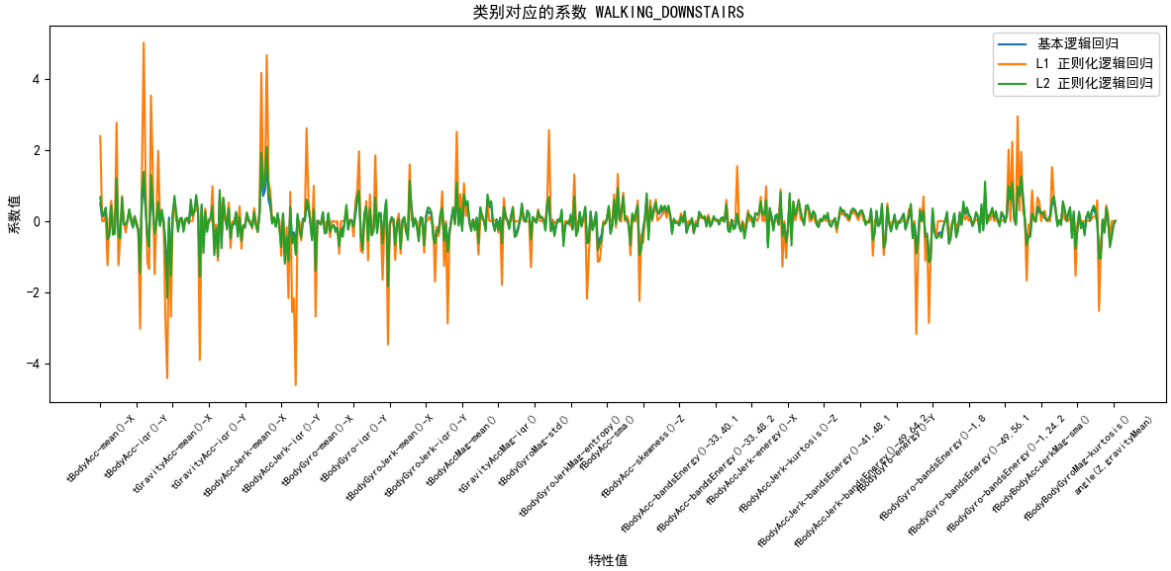
```python
# 绘制每个类别的特征系数图
for i in range(num_classes):
    plt.figure(figsize=(12, 6))
    plt.plot(lr_coef.iloc[i], label='基本逻辑回归')
    plt.plot(lr_l1_coef.iloc[i], label='L1 正则化逻辑回归')
    plt.plot(lr_l2_coef.iloc[i], label='L2 正则化逻辑回归')
    plt.title(f'类别对应的系数 {le.classes_[i]}')
    plt.xlabel('特性值')
    plt.ylabel('系数值')
    plt.legend()
   # 间隔选取 x 轴标签
    x_ticks = np.arange(0, len(feature_cols), 20)  # 每隔 20 个取一个
    x_labels = [feature_cols[j] for j in x_ticks]
    plt.xticks(x_ticks, x_labels, rotation = 45, fontsize = 8)

    plt.tight_layout()
    plt.show()
```

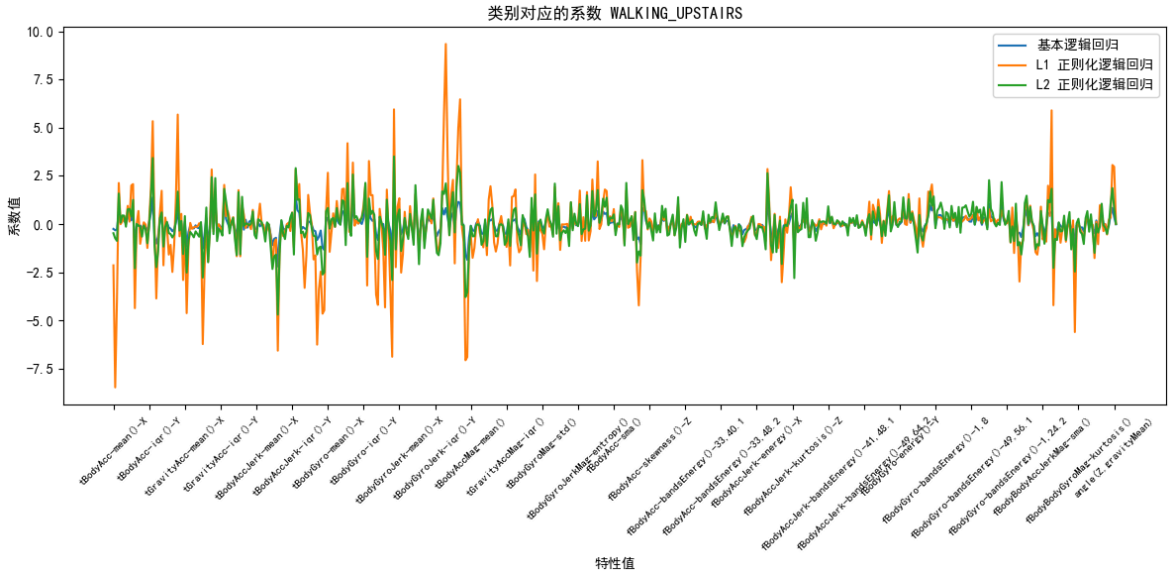





类别对应的系数 STANDING
基本逻辑回归
L1 正则化逻辑回归
L2 正则化逻辑回归
系数值
特性值


类别对应的系数 WALKING
基本逻辑回归
L1 正则化逻辑回归
L2 正则化逻辑回归
系数值
特性值


类别对应的系数 WALKING_DOWNSTAIRS
基本逻辑回归
L1 正则化逻辑回归
L2 正则化逻辑回归
系数值
特性值



第五步：预测数据

- 将每个模型预测的类别和概率值都保存下来。

在 [27]…

```
# 请在此处填写你的代码
# 基本逻辑回归模型预测
lr_pred = lr.predict(X_test)
lr_proba = lr.predict_proba(X_test)

# L1 正则化逻辑回归模型预测
lr_l1_pred = lr_l1.predict(X_test)
lr_l1_proba = lr_l1.predict_proba(X_test)

# L2 正则化逻辑回归模型预测
lr_l2_pred = lr_l2.predict(X_test)
lr_l2_proba = lr_l2.predict_proba(X_test)

# 保存预测结果
predictions = {
    'Basic Logistic Regression': {
        'predictions': lr_pred,
        'probabilities': lr_proba
    },
    'L1 Regularized Logistic Regression': {
        'predictions': lr_l1_pred,
        'probabilities': lr_l1_proba
    },
    'L2 Regularized Logistic Regression': {
        'predictions': lr_l2_pred,
        'probabilities': lr_l2_proba
    }
}
```